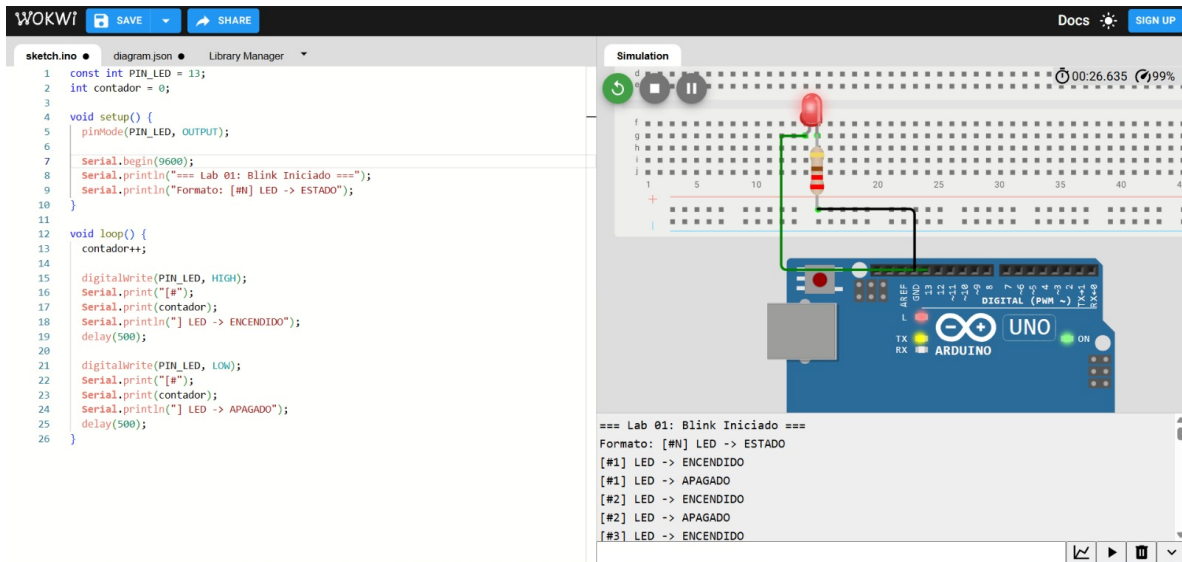


Laboratorio 1



Programa mostrando “ENCENDIDO” y “APAGADO”

Se cargo el código proporcionado en el manual de prácticas para su posterior modificación en las imágenes mostradas mas adelante en el documento

Ejercicio 1

The screenshot displays the Wokwi online Arduino IDE interface. On the left, the 'sketch.ino' file contains the following code:

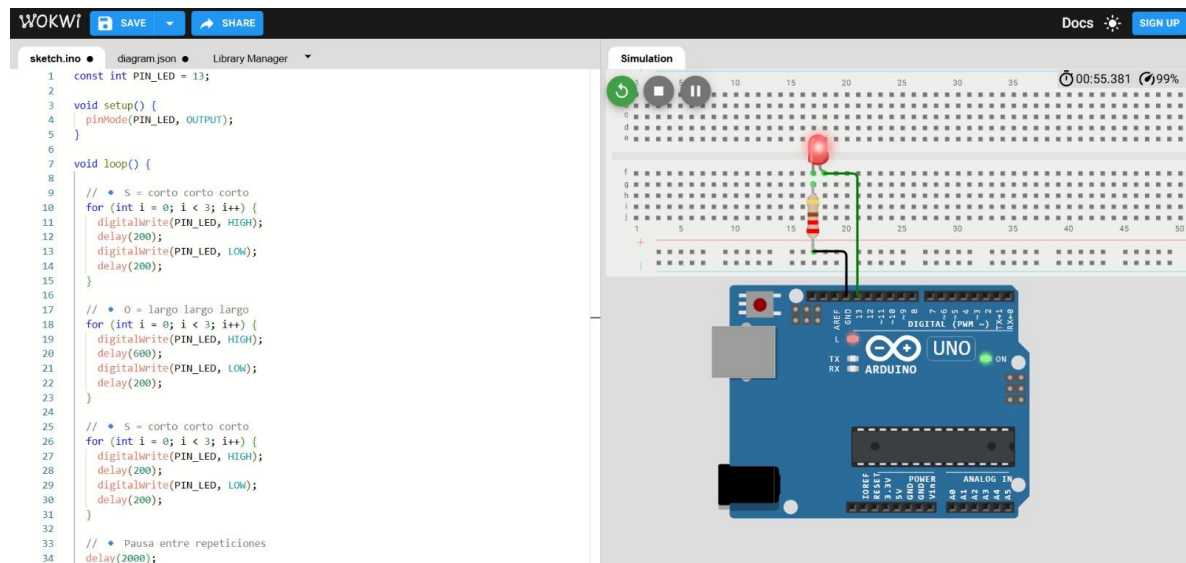
```
1 const int PIN_LED = 13;
2 int contador = 0;
3
4 void setup() {
5   pinMode(PIN_LED, OUTPUT);
6   Serial.begin(9600);
7   Serial.println("=== Lab 01: Blink Iniciado ===");
8   Serial.println("Formato: [#N] LED -> ESTADO");
9 }
10
11 void loop() {
12   contador++;
13
14   digitalWrite(PIN_LED, HIGH);
15   Serial.print("#");
16   Serial.print(contador);
17   Serial.println(" LED -> ENCENDIDO");
18   delay(200); // Encendido 200 ms
19
20   digitalWrite(PIN_LED, LOW);
21   Serial.print("#");
22   Serial.print(contador);
23   Serial.println(" LED -> APAGADO");
24   delay(800); // Apagado 800 ms
25 }
26
```

On the right, the 'Simulation' window shows a virtual Arduino Uno board with a red LED connected to pin 13. The LED is currently lit. Below the board, the serial monitor displays the output of the sketch:

```
=== Lab 01: Blink Iniciado ===
Formato: [#N] LED -> ESTADO
[#1] LED -> ENCENDIDO
[#1] LED -> APAGADO
[#2] LED -> ENCENDIDO
[#2] LED -> APAGADO
[#3] LED -> ENCENDIDO
```

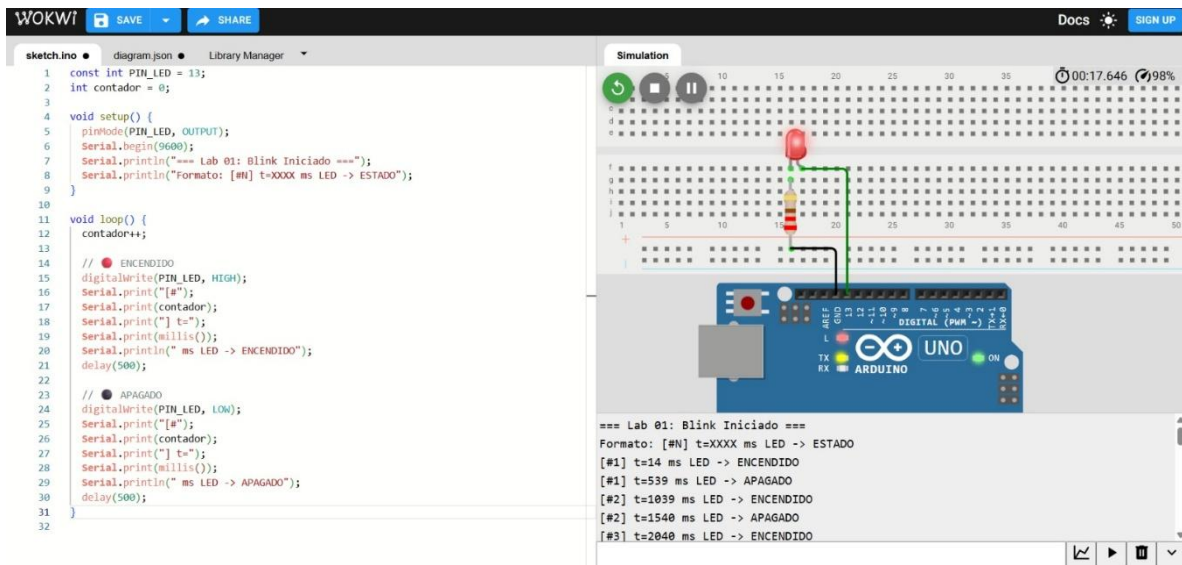
En este ejercicio se modificó el delay (retraso) que tienen los led, a 200 y 800 ms respectivamente esto con el fin de notar como cambia el ritmo con el que se muestran los mensajes de led encendido y led apagado

Ejercicio 2



En este ejercicio se modificó el código para que los mensajes de led encendido y apagado se muestren en diferentes periodos de tiempo, así se pudo comunicar un mensaje de SOS en código morse, se creó un método en bucle para determinar cuanto tiempo se mostrara cada mensaje como lo indica las instrucciones de la practica

Ejercicio 3

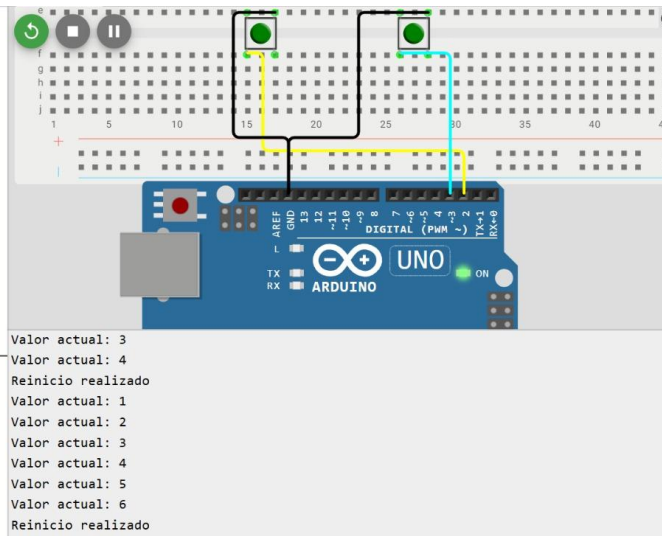


En esta última parte de la practica se modificó el código para mostrar los resultados serial en la consola, mostrando los milisegundos que han transcurrido desde que se ejecuto el programa y los seguirá mostrando hasta su detención o interrupción

Laboratorio 2

Ejercicio1

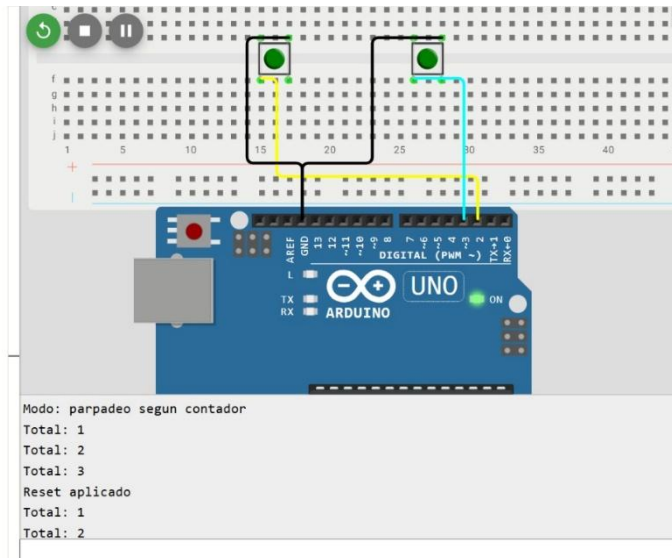
```
1  const int BTN_CONT = 2;
2  const int BTN_RESET = 3;
3  const int LED = 13;
4
5  int valor = 0;
6
7  bool estadoPrevCont = HIGH;
8  bool estadoPrevReset = HIGH;
9
10 void setup() {
11   pinMode(BTN_CONT, INPUT_PULLUP);
12   pinMode(BTN_RESET, INPUT_PULLUP);
13   pinMode(LED, OUTPUT);
14
15   Serial.begin(9600);
16   Serial.println("Inicio del sistema - contador = 0");
17 }
18
19 void loop() {
20
21   bool estadoCont = digitalRead(BTN_CONT);
22   bool estadoReset = digitalRead(BTN_RESET);
23
24   // --- BOTÓN CONTADOR ---
25   if (!estadoCont && estadoPrevCont) {
26     delay(50);
27     valor++;
28
29     Serial.print("Valor actual: ");
30     Serial.println(valor);
31
32     // Feedback LED
33     digitalWrite(LED, HIGH);
34 }
```



El código implementa un sistema de conteo con dos botones usando resistencias internas pull-up, donde uno incrementa un contador y el otro lo reinicia, mostrando el valor en el monitor serial y utilizando un LED como retroalimentación visual, además de emplear detección de flanco y anti-rebote para evitar lecturas erróneas.

Ejercicio 2

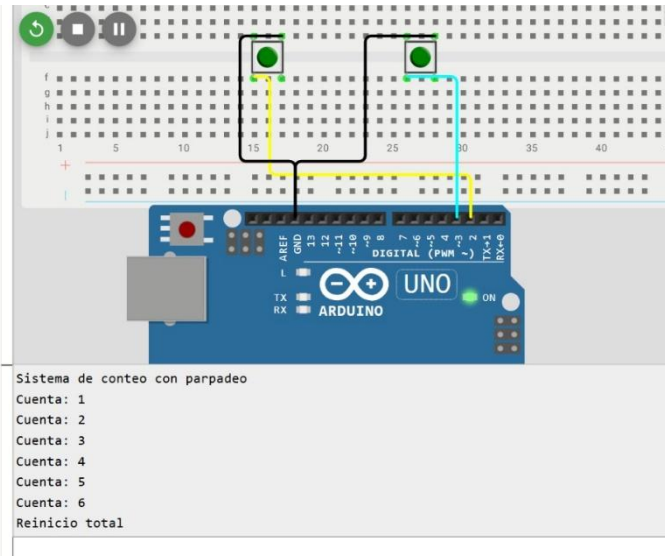
```
1  const int BTN_SUMA = 2;
2  const int BTN_CLEAR = 3;
3  const int LED_PIN = 13;
4
5  int conteo = 0;
6
7  bool prevSuma = HIGH;
8  bool prevClear = HIGH;
9
10 void setup() {
11   pinMode(BTN_SUMA, INPUT_PULLUP);
12   pinMode(BTN_CLEAR, INPUT_PULLUP);
13   pinMode(LED_PIN, OUTPUT);
14
15   Serial.begin(9600);
16   Serial.println("Modo: parpadeo segun contador");
17 }
18
19 void loop() {
20
21   bool estadoSuma = digitalRead(BTN_SUMA);
22   bool estadoClear = digitalRead(BTN_CLEAR);
23
24   // ---- BOTÓN SUMA ----
25   if (!estadoSuma && prevSuma) {
26     delay(50);
27
28     conteo++;
29
30     Serial.print("Total: ");
31     Serial.println(conteo);
32
33     // Parpadeo proporcional
34     for (int k = 0; k < conteo; k++) {
```



Este código implementa un sistema de conteo con dos botones usando resistencias pull-up internas, donde uno incrementa el contador y provoca que un LED parpadee tantas veces como el valor acumulado, mientras que el otro botón reinicia el contador a cero mostrando el cambio en el monitor serial y utilizando un parpadeo largo del LED como confirmación, además de incluir detección de flanco y anti-rebote para evitar errores en la lectura.

Ejercicio 3

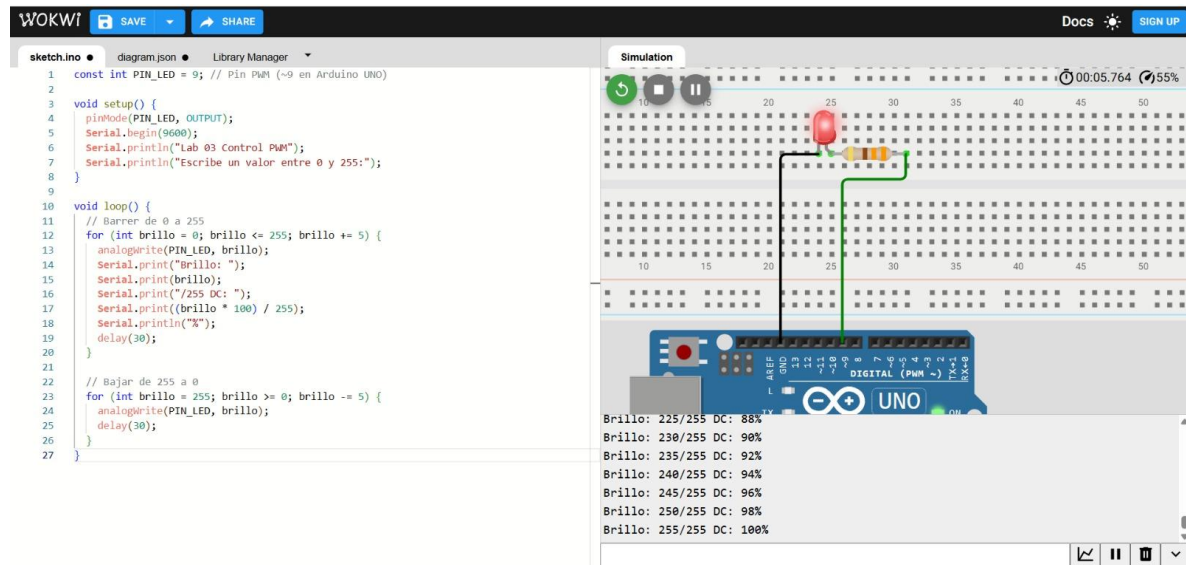
```
1  const int BTN_UP = 2;  
2  const int BTN_RST = 3;  
3  const int LED_OUT = 13;  
4  
5  int valorActual = 0;  
6  
7  bool lastUp = HIGH;  
8  bool lastRst = HIGH;  
9  
10 void setup() {  
11   pinMode(BTN_UP, INPUT_PULLUP);  
12   pinMode(BTN_RST, INPUT_PULLUP);  
13   pinMode(LED_OUT, OUTPUT);  
14  
15   Serial.begin(9600);  
16   Serial.println("Sistema de conteo con parpadeo");  
17 }  
18  
19 void loop() {  
20  
21   bool lecturaUp = digitalRead(BTN_UP);  
22   bool lecturaRst = digitalRead(BTN_RST);  
23  
24   // ----- SUMAR -----  
25   if (lecturaUp == LOW && lastUp == HIGH) {  
26     delay(50);  
27  
28     valorActual += 1;  
29  
30     Serial.print("Cuenta: ");  
31     Serial.println(valorActual);  
32  
33     ejecutarParpadeo(valorActual);  
34   }
```



Este código implementa un sistema de conteo con dos botones utilizando resistencias pull-up internas, donde uno incrementa el valor almacenado y activa una función que hace parpadear un LED tantas veces como el número actual, mientras que el otro botón reinicia el contador a cero mostrando el resultado en el monitor serial y utilizando una señal visual, incorporando además detección de flanco y anti-rebote para garantizar lecturas correctas.

Laboratorio 3

Ejercicio1



El brillo inicializa en 0, se usa un for que tiene una variable que se llama brillo la cual realiza una comparación, tiene un incremento que va de cinco en cinco y lo guarda, a medida que aumenta el numero incrementa el brillo del led, el otro lo que hace es decrementar el brillo que recibe el led

Ejercicio 2

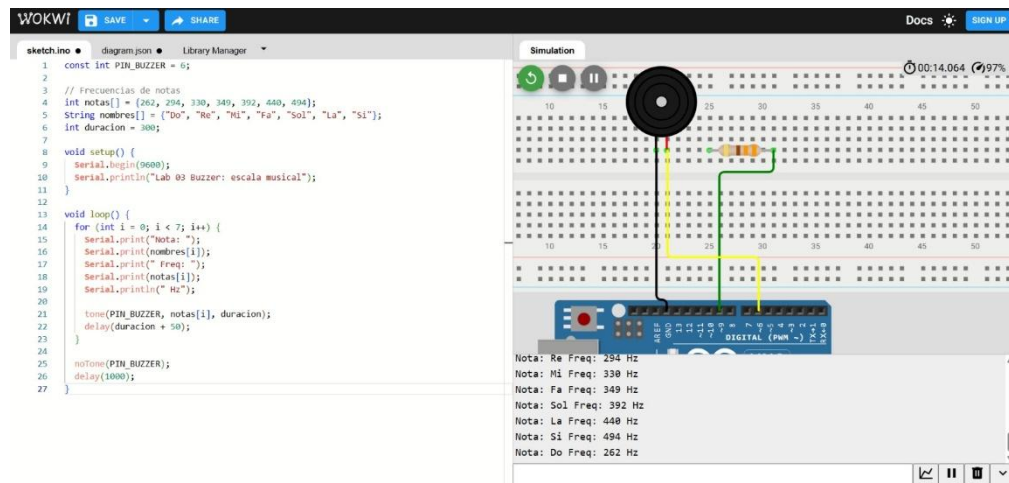
The screenshot shows the WOKWI simulation interface. On the left, the code for sketch.ino is displayed:

```
1 #include <math.h>
2
3 const int PIN_LED = 9;
4 float angulo = 0.0;
5
6 void setup() {
7   pinMode(PIN_LED, OUTPUT);
8 }
9
10 void loop() {
11   // Convertimos seno (-1 a 1) a (0 a 1)
12   float seno = (sin(angulo) + 1.0) / 2.0;
13
14   // Escalamos a 0-255
15   int brillo = (int)(seno * 255);
16
17   analogWrite(PIN_LED, brillo);
18
19   angulo += 0.05; // Velocidad
20
21   if (angulo > 2 * PI) angulo = 0;
22
23   delay(15);
24 }
```

On the right, the simulation shows an Arduino Uno board connected to a breadboard. A red LED is connected to digital pin 9 (via a 220Ω resistor) and ground. The simulation status bar at the top right shows a timer at 00:14.194 and 59% battery. The bottom right corner has icons for zoom, pause, and delete.

Utiliza un valor matemático que es Sen, utiliza el angulo el cual se ira sumando en 1 que después se dividirá en 2 para causar una subida de brillo mas suave, se hace un mapeo matemático desde el 0.0 al 1.0 y con el seno devuelve un valor entre 0.0 y 1.0, se guarda en una variable llamada seno, al final el resultado se convierte en entero y se guarda en brillo, Reinicia el ciclo cuando realiza la vuelta completa.

Ejercicio 3



Se unen 2 variables hay un int que se convertirá en un arreglo que va a tomar las frecuencias de las notas y string tomara el nombre de las frecuencias se encuentran dentro de un arreglo, tendrá otra variable de tipo entero que es la duración en ms de la nota

Donde se encuentra conectado dará un pulso agarrándolo desde el arreglo de las notas con una duración de 360 ms que es la cantidad que seguirá sonando el sonido, y hace una pausa entre escalas, quiere decir que hace una pausa de un segundo entero